FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Angelo Morzenti

Exam of FRIDAY 20 JANUARY 2023 – Theory Part

WITH SOLUTIONS – FOR TEACHING PURPOSES THE SOLUTIONS ARE WIDELY COMMENTED

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA: SIGNATURE:

(or PERSON CODE)

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. A. MORZENTI

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
  1. Theory (80%): syntax and semantics of languages, divided in four sections:
     - regular expressions and finite automata
     - free grammars and pushdown automata
     - syntax analysis and parsing methodologies
     - language translation and semantic analysis
  2. Lab (20%): compiler design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- For the theory part to be valid, the candidate must achieve a grade of at least 10/20 on each of the four sections.
- Correctly answering all the questions that are not marked optional, allows the candidate to achieve a high grade (although not the full mark).
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30.
- The final grade is the weighted average of the theory part (80 %) and of the lab part (20 %), and must be $\geq 18/30$.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

1 Regular Expressions and Finite Automata 20%

1. Consider the regular expression R below, over the two-letter alphabet $\Sigma = \{ a, b \}$:

$$R = (a \mid b\, a \mid a\, b)^*$$

Answer the following questions:

(a) Write all the strings of length exactly 3 that are generated by expression R.

(b) Prove that expression R is ambiguous: to this end provide the shortest ambiguous string $x \in L(R)$ and show why string x is ambiguous.

(c) Design a finite-state automaton A that accepts language $L(R)$, by using the Berry-Sethi (BS) method.

(d) Check if the BS deterministic automaton A designed at point (c) is minimal, and if it is not, minimize it.

(e) Starting from the BS automaton A (possibly in the minimized version), design an automaton A' that accepts the complement language of $L(A)$.

(f) (optional) By using the sets of the followers identified for the design of the BS automaton A, argue that no string in the language $L(R)$ can contain three consecutive occurrences of letter b.

Solution

(a) Here are all the four strings of length 3 that are generated by expression R, alphabetically listed ($a < b$):

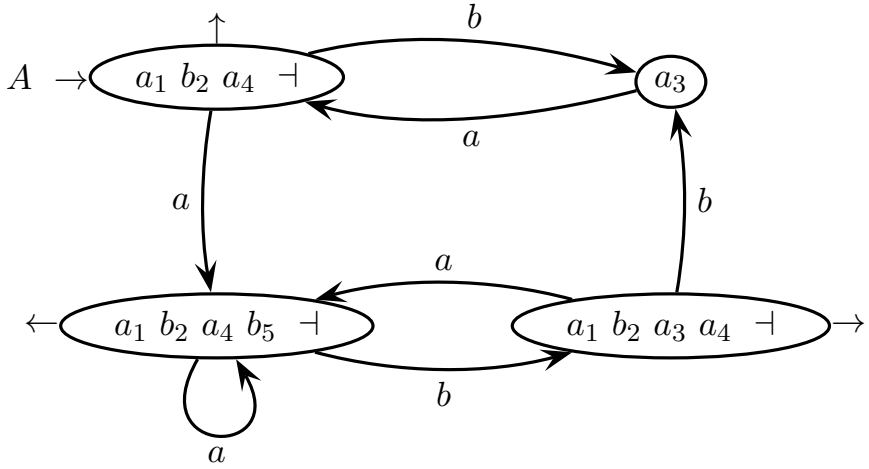
$$a\,a\,a \qquad a\,a\,b \qquad a\,b\,a \qquad b\,a\,a$$

(b) The numbered expression $R_{\#} = (\, a_1 \;\mid\; b_2\; a_3 \;\mid\; a_4\; b_5\,)^*$ can generate the two (numbered) strings $a_1\,b_2\,a_3$ and $a_4\,b_5\,a_1$, which both map to the string $a\,b\,a$ generated by expression R. Therefore expression R is ambiguous.

(c) Here is the application of the Berry-Sethi (BS) method to expression R, to obtain a deterministic finite-state automaton A equivalent to R.

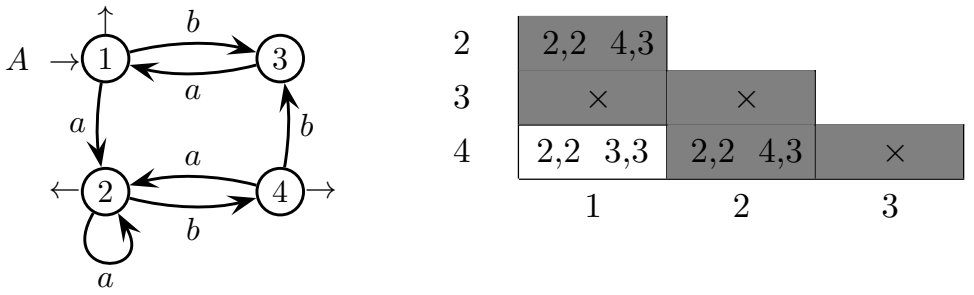
$$R_{\#} = (\, a_1 \;\mid\; b_2\; a_3 \;\mid\; a_4\; b_5\,)^*$$

initials	$a_1\ b_2\ a_4\ \dashv$
terminal	**followers**
a_1	$a_1\ b_2\ a_4\ \dashv$
b_2	a_3
a_3	$a_1\ b_2\ a_4\ \dashv$
a_4	b_5
b_5	$a_1\ b_2\ a_4\ \dashv$

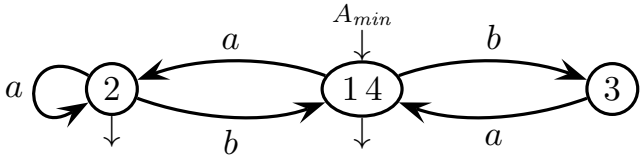


The BS automaton A equivalent to expression R is deterministic by construction, and has four states. It may not be minimal, anyway.

(d) Here is the application of the Nerode method (state undistinguishability) to the (deterministic) automaton A (after renaming the states) for state minimization:

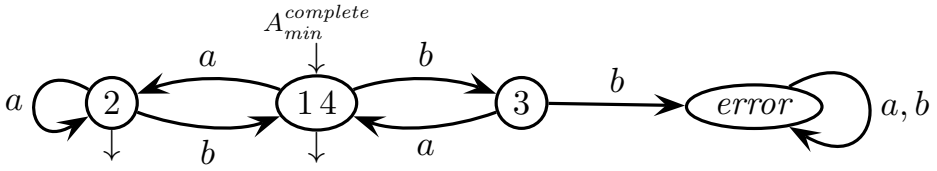


The (final) states 1 and 4 are undistinguishable and can be merged, as follows:

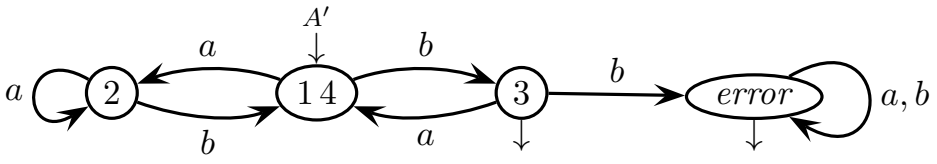


There are no other pairs of undistinguishable states. Therefore the minimal automaton A_{min} of expression R has three states.

(e) Here is the (deterministic) automaton A' that accepts the complement language of $L(R)$, obtained from the minimal automaton A_{min}. First, the minimal automaton has to be naturally completed with an error state, as follows:



Then, the complement automaton A' is obtained by switching the non-final and final states of automaton $A_{min}^{complete}$, as follows:



Of course, the (non-final) states 14 and 2 are still distinguishable, as before, and it is easy to see that the (final) states 3 and *error* are distinguishable, too. Therefore automaton A' happens to be already minimal.

(f) From the follower sets of expression $R_{\#}$, letter b_5 can be followed by letter b_2, which in turn can be followed only by letter a_3. No other numbered occurrences of letter b are possible, therefore the maximum number of consecutive occurrences of letter b for expression R is two, corresponding to the numbered substring $b_5\, b_2$.

2 Free Grammars and Pushdown Automata 20%

1. Consider the following two languages L_1 and L_2, over the three-letter and two-letter alphabets $\Sigma_1 = \{ a,\ b,\ c \}$ and $\Sigma_2 = \{ a,\ c \}$, respectively:

$$L_1 = \left\{ a^n\ b^m\ c^k \mid \quad n + m < k, \quad n \geq 0, \quad m \geq 0, \quad k > 0 \right\}$$

$$L_2 = \left\{ a^n\ c^k \mid \quad n > 0, \quad k > 0 \right\}$$

   Answer the following questions (all the requested grammars must be BNF):

   (a) Write a grammar G_1 for language L_1.

   (b) Draw the syntax tree for the valid string below:

$$a\,b\,c\,c\,c \in L_1$$

   (c) Write a grammar $G_\cap$ for the intersection language $L_1 \cap L_2$.

   (d) Write a grammar $G_\cup$ for the union language $L_1 \cup L_2$.

   (e) (optional) Is any of the three languages L_1, $L_1 \cap L_2$ and $L_1 \cup L_2$ inherently ambiguous?

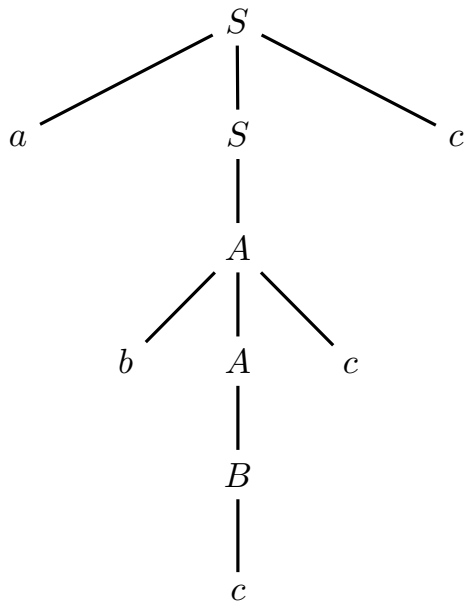
Solution

(a) Here is a grammar G_1 for language L_1 (axiom S):

$$G_1 \begin{cases} S \;\to\; a\,S\,c \;\mid\; A \\ A \;\to\; b\,A\,c \;\mid\; B \\ B \;\to\; c\,B \;\mid\; c \end{cases}$$

Grammar G_1 first generates a nested structure $a^n\,b^m\,c^k$, where the nonterminals S and A respectively generate in succession the components with $n \geq 0$ letters a and $m \geq 0$ letters b, thus yielding the equality $n + m = k$. Then grammar G_1 puts additional letters c (at least one) in the string centre, thus yielding the inequality $n+m < k$ (with $k > 0$), and eventually terminates the derivation. See also the sample syntax tree at point (b). Clearly grammar G_1 is not ambiguous, as how many rules should be used in a derivation, and in what order such rules should be applied, is determined by the final string, in only one way.

(b) Here is the syntax tree of the valid string $a\,b\,c\,c\,c \in L_1$, for grammar G_1:



As observed at point (a), grammar G_1 is non-ambiguous, thus this tree is unique.

(c) Here is a grammar $G_\cap$ for the intersection language $L_1 \cap L_2$ (axiom S):

$$G_\cap \begin{cases} S \;\to\; a\,S\,c \;\mid\; a\,B\,c \\ B \;\to\; c\,B \;\mid\; c \end{cases}$$

Notice that it is $L_2 = L\,(a^+\,c^+)$, therefore the intersection forces to have exponents $n > 0$ and $m = 0$ in L_1, thus it also holds $n < k$ and consequently it is $L_1 \cap L_2 = \left\{\, a^n c^k \mid \quad n < k,\ n > 0,\ k > 0 \,\right\}$. Therefore the grammar $G_\cap$ above is quickly obtained as a straightforward modification of grammar G_1, by suppressing the generation of letters b and consequently removing nonterminal A. After such simple operations on G_1, the resulting grammar $G_\cap$ is still non-ambiguous. The shortest string generated by $G_\cap$ is $a\,c\,c$, since it holds $0 < n < k$.

(d) Here is a grammar $G_\cup$ for the union language $L_1 \cup L_2$ (axiom S_3):

$$G_\cup \left\{ \begin{array}{lll} S_3 \rightarrow S_1 \mid S_2 & \text{// axiomatic rule} \\ \hline S_1 \rightarrow a\, S_1\, c \mid A & \text{// grammar } G_1 \\ A \rightarrow b\, A\, c \mid B & \\ B \rightarrow c\, B \mid c & \\ \hline S_2 \rightarrow a\, S_2 \mid a\, C & \text{// grammar } G_2 \\ C \rightarrow c\, C \mid c & \end{array} \right.$$

For language L_2, which is regular (see point (c)), a grammar G_2 is immediate to obtain (for instance a right-unilinear grammar). Grammar $G_\cup$ results then from the standard union construction of grammars G_1 (see point (a)) and G_2.

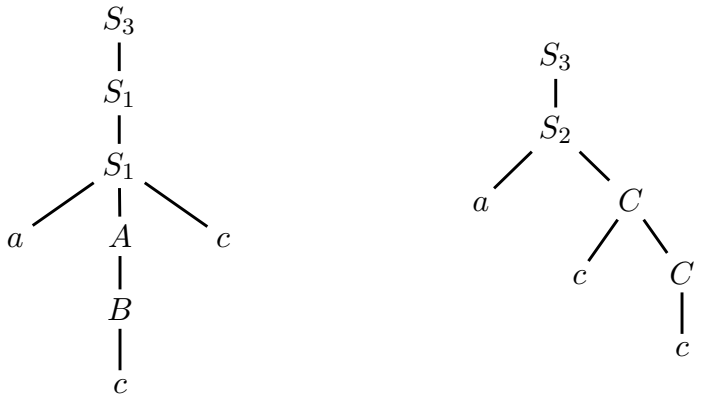
(e) Grammars G_1 and $G_\cap$ are not ambiguous, so their languages L_1 and $L_1 \cap L_2$ are not inherently ambiguous. The simple formulation (by means of the standard union construction, see point (d)) of grammar $G_\cup$ is ambiguous, because for instance string $a\,c\,c$ has two derivations:

$$S_3 \Rightarrow S_1 \Rightarrow a\, S_1\, c \Rightarrow a\, A\, c \Rightarrow a\, B\, c \Rightarrow a\, c\, c$$

and

$$S_3 \Rightarrow S_2 \Rightarrow a\, C \Rightarrow a\, c\, C \Rightarrow a\, c\, c$$

or equivalently such a string has two different syntax trees:



We can reformulate $L_1 \cup L_2 = (L_1 - L_2) \cup (L_2 - L_1) \cup (L_1 \cap L_2)$ instead, i.e., as the union of three component languages that are disjoint by definition. This is not inherently ambiguous if all the components are not inherently ambiguous. It is already known that $L_1 \cap L_2$ is not inherently ambiguous, as said before, but the two difference languages $L_1 - L_2$ and $L_2 - L_1$ have to be examined closely. Language $L_2 - L_1$ can be expressed as $\{\, a^n\, c^k \mid \quad n > 0,\ k > 0,\ n \geq k \,\}$, which is trivially context-free and not inherently ambiguous. Language $L_1 - L_2$ can be expressed as the union $\{\, a^n\, b^m\, c^k \mid \quad n + m < k,\ m > 0 \,\} \cup \{\, c^k \mid \quad k > 0 \,\}$, and these two sublanguages are disjoint and not inherently ambiguous, so $L_1 - L_2$ is not inherently ambiguous. Thus the union grammar of the three components above is non-ambiguous. Therefore $L_1 \cup L_2$ is not inherently ambiguous either. Different decompositions of the union language $L_1 \cup L_2$ into disjoint components may exist, possibly by exploiting the characteristics of the languages involved.

2. Consider a fragment of the declarative section of a program, written in a C-like language with some restrictions, which has the following features:

   - the declarative section is a non-empty list of variable declarations, which are terminated by a semicolon
   - the variable types are *structure*, *union* and *scalar* (as in the C language)
   - a structure must contain one or more variable declarations, i.e., fields
   - a union must contain two or more variable declarations, i.e., fields, but only of union or scalar type (unions may not contain structures, at any level)
   - the scalar types are *character* and *integer*
   - a scalar variable can be initialized (with a constant) in the declaration itself

   For the remaining details of the declarations sketched above, and the lexical tokens (keywords, symbols, etc), see the example below (the comments aside, introduced by `//`, are not part of the language):

```
struct OMEGA {                          // one struct with three fields
    char alpha ;                        // one scalar
    union {                             // one union with two fields
        char gamma = 'g' ;              // one scalar, initialized
        union DELTA {                   // one union with two fields
            int one = 1, two ;          // two scalars, one initialized
            char three ;                // one scalar
        } comp ;
    } beta ;
    struct {                            // two structs with one field
        int a1 = 3 ;                    // one scalar, initialized
    } s1, s2 ;
} var ;
char cc1, cc2 = 'a' ;                   // two scalars, one initialized
```

   Answer the following questions:

   (a) Write an EBNF grammar, non-ambiguous, that generates the language fragment sketched above. The identifiers can be schematized by the terminal id, and the character and integer constants can be schematized by the terminals cc and ii, respectively, which do not need to be further expanded.

   (b) Draw the syntax tree of the sample language fragment shown below:

```
char alpha ;                            // one scalar
struct {                                // one struct with two fields
    char gamma = 'g' ;                  // one scalar, initialized
    union DELTA {                       // one union with two fields
        int one = 1, two ;              // two scalars, one initialized
        char three ;                    // one scalar
    } comp ;
} beta ;
```

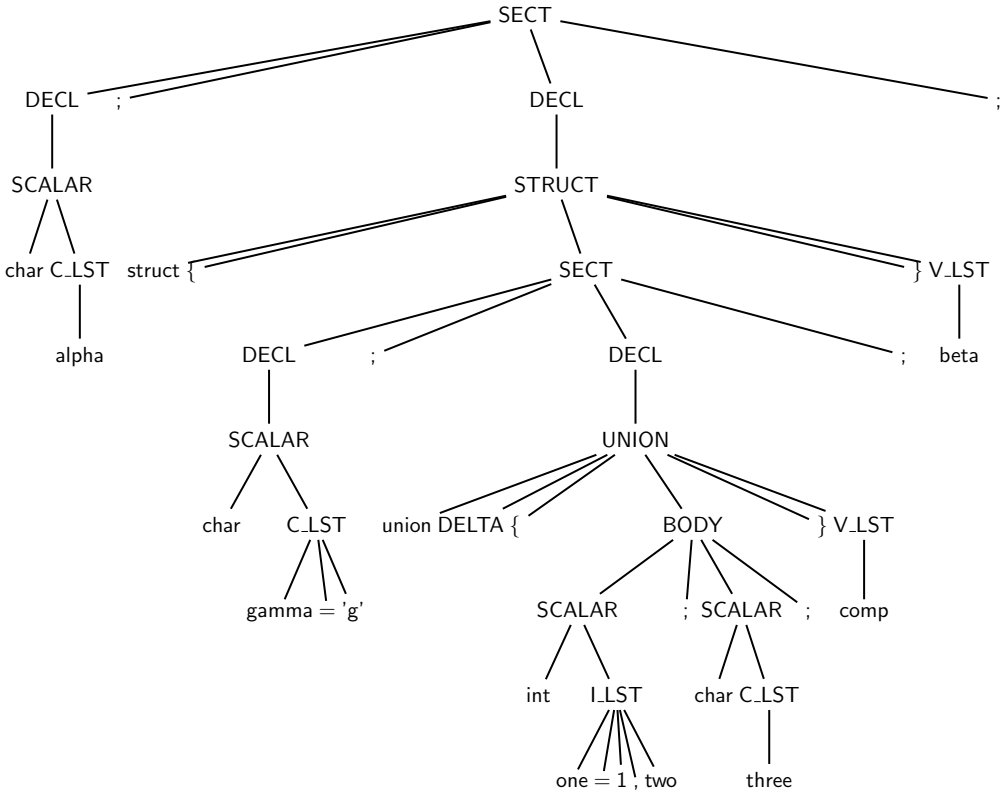
Solution

(a) Here is an EBNF grammar for the sketched language fragment (axiom SECT):

$$
\left\{
\begin{array}{rcl}
\langle \mathsf{SECT} \rangle & \to & \big(\langle \mathsf{DECL} \rangle \ \text{“;”} \big)^{+} \\
\hline
\langle \mathsf{DECL} \rangle & \to & \langle \mathsf{STRUCT} \rangle \mid \langle \mathsf{UNION} \rangle \mid \langle \mathsf{SCALAR} \rangle \\
\hline
\langle \mathsf{STRUCT} \rangle & \to & \mathsf{struct} \ \big[\, \mathsf{id} \, \big] \ \text{“\{”} \ \langle \mathsf{SECT} \rangle \ \text{“\}”} \ \langle \mathsf{V\_LST} \rangle \\
\langle \mathsf{UNION} \rangle & \to & \mathsf{union} \ \big[\, \mathsf{id} \, \big] \ \text{“\{”} \ \langle \mathsf{BODY} \rangle \ \text{“\}”} \ \langle \mathsf{V\_LST} \rangle \\
\langle \mathsf{SCALAR} \rangle & \to & \mathsf{char} \ \langle \mathsf{C\_LST} \rangle \mid \mathsf{int} \ \langle \mathsf{I\_LST} \rangle \\
\hline
\langle \mathsf{BODY} \rangle & \to & \big[\big(\langle \mathsf{UNION} \rangle \mid \langle \mathsf{SCALAR} \rangle \big) \ \text{“;”} \big]_2^{*} \\
\hline
\langle \mathsf{V\_LST} \rangle & \to & \mathsf{id} \ \big(\text{“,”} \ \mathsf{id} \big)^{*} \\
\langle \mathsf{C\_LST} \rangle & \to & \mathsf{id} \ \big[\text{“=”} \ \mathsf{cc} \big] \ \big(\text{“,”} \ \mathsf{id} \ \big[\text{“=”} \ \mathsf{cc} \big] \big)^{*} \\
\langle \mathsf{I\_LST} \rangle & \to & \mathsf{id} \ \big[\text{“=”} \ \mathsf{ii} \big] \ \big(\text{“,”} \ \mathsf{id} \ \big[\text{“=”} \ \mathsf{ii} \big] \big)^{*}
\end{array}
\right.
$$

The operator $[\,a\,]_2^*$ is non-standard and means “repeat the argument at least twice”, i.e., $a\,a$, $a\,a\,a$, …, and is equivalent to the standard formulation $a^2\,a^*$, therefore it is a regular operator as well. This grammar is not ambiguous.

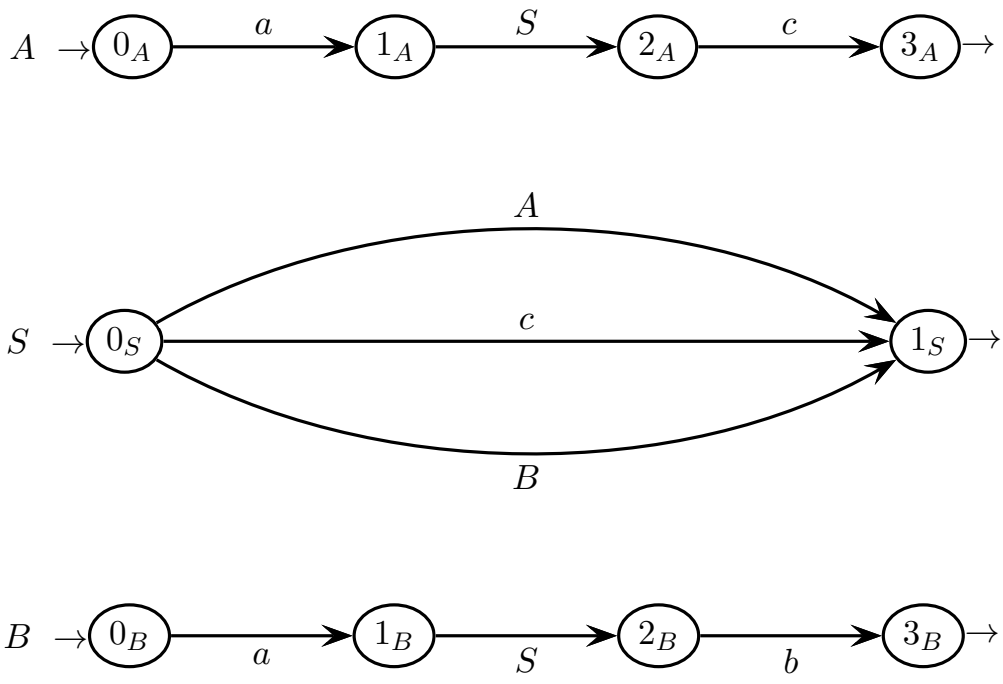
(b) Here is the syntax tree of the sample declaration fragment:



All the grammar rules are used. For better understanding, the terminals are expanded to the actual names they have in the sample code fragment.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, represented as a machine net, over the three-letter terminal alphabet $\Sigma = \{ a, b, c\}$ and the three-symbol nonterminal alphabet $V_N = \{ S, A, B \}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

(a) Draw the syntax tree of the valid string below:

$$a\,a\,c\,c\,b$$

(b) Prove that grammar G is ELR (1) by drawing the pilot automaton of G (complete the draft already given) and showing that the pilot is free of any conflict.

(c) Write a simulation of the bottom-up analysis of the valid string below:

$$a\,a\,c\,c\,b$$

(d) Prove that grammar G is not ELL (1) by considering its guide sets.

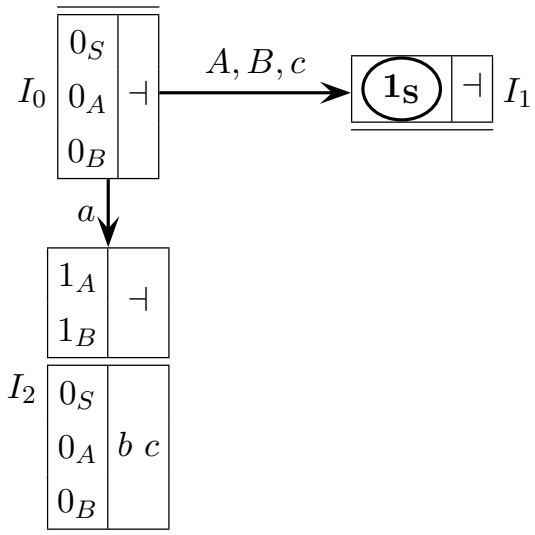
(e) Say if grammar G is ELL (k) for some k. Suitably justify your answer.

(f) Design an equivalent grammar G' that has only one nonterminal symbol (the axiom S) and is ELL (1). Represent grammar G' as a machine net (with just one machine M_S) and draw on such a machine all the relevant guide sets.

(g) (optional) Write the recursive-descent procedure for the top-down analysis of grammar G'.

question (a) – syntax tree of the valid string $a\,a\,c\,c\,b$

question (b) – draft pilot of grammar G to be completed



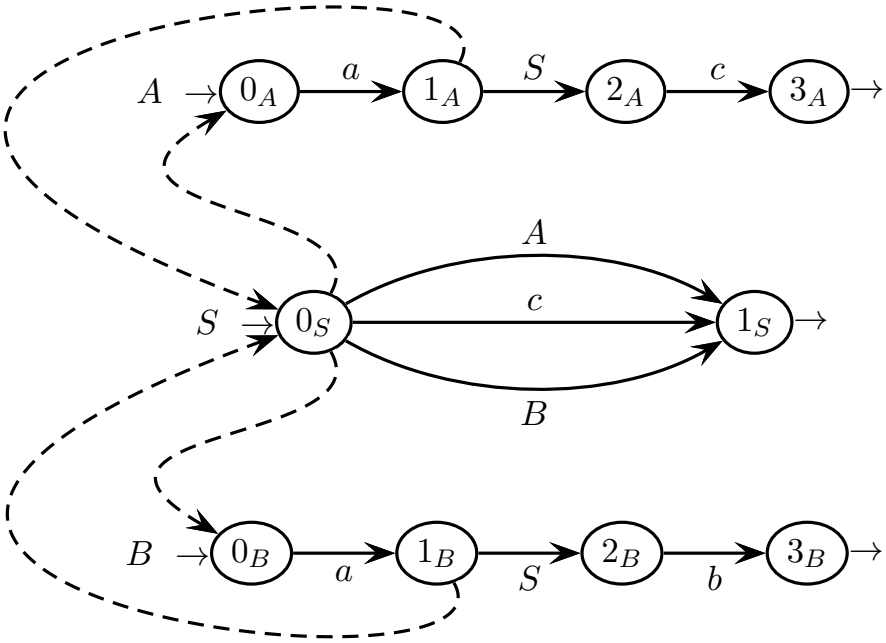
question (c) – ELR simulation table

string $a\,a\,c\,c\,b$ – number of rows not significant

stack of m-states and grammar symbols	*input left to scan*	*move executed*
I_0	$a\ a\ c\ c\ b\ \dashv$	initial configuration
$I_0\ a\ I_2$	$a\ c\ c\ b\ \dashv$	terminal shift: $I_0 \xrightarrow{a} I_2$

tree building steps

question (d) – guide sets (on all the call arcs and exit arrows) and answer



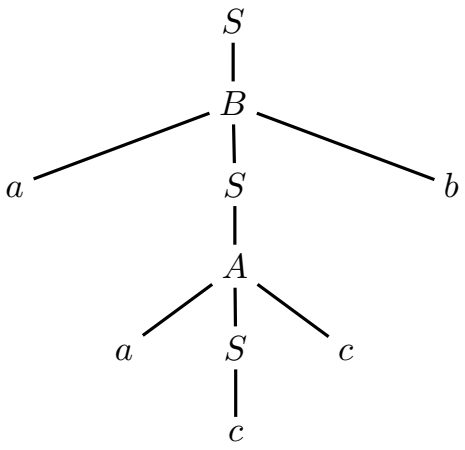
question (e) – space for answering (about ELL (k))

question (f) – space for answering (single machine of G' and guide sets)

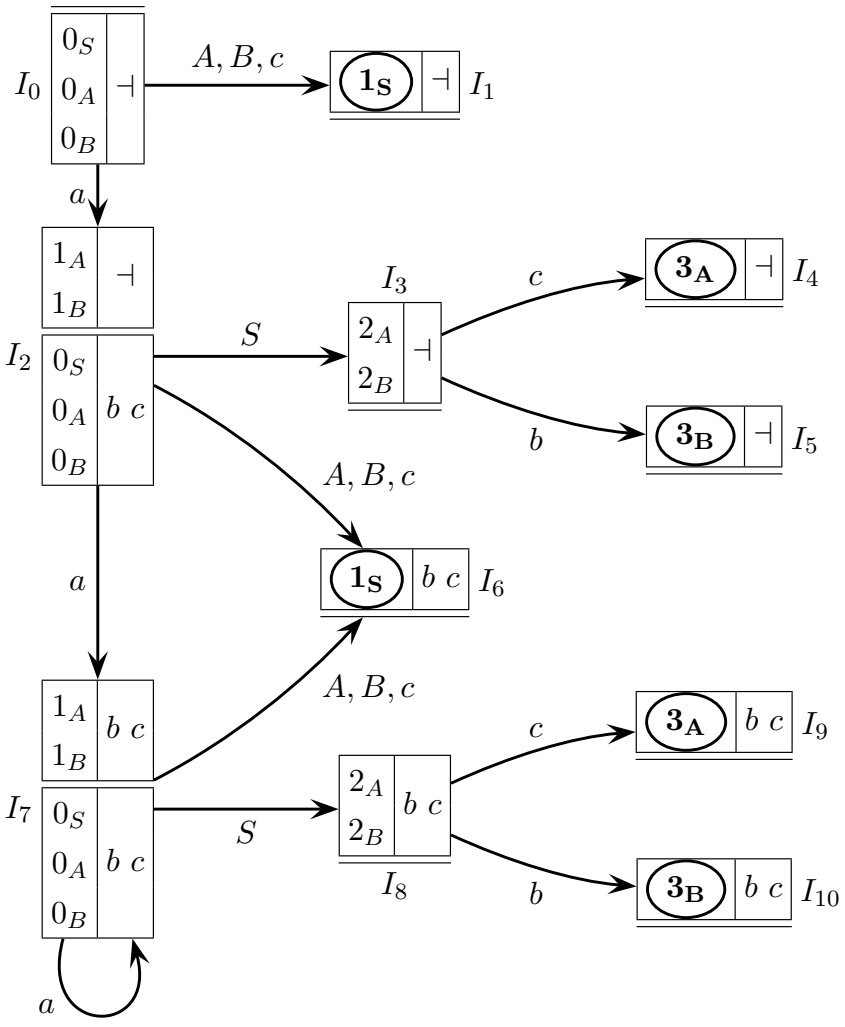
question (g) – space for answering (recursive-descent analyzer of G')

Solution

(a) Here is the (unique) syntax tree of the (valid) string $a\,a\,c\,c\,b$:



(b) Here is the pilot automaton of grammar G, with eleven m-states:



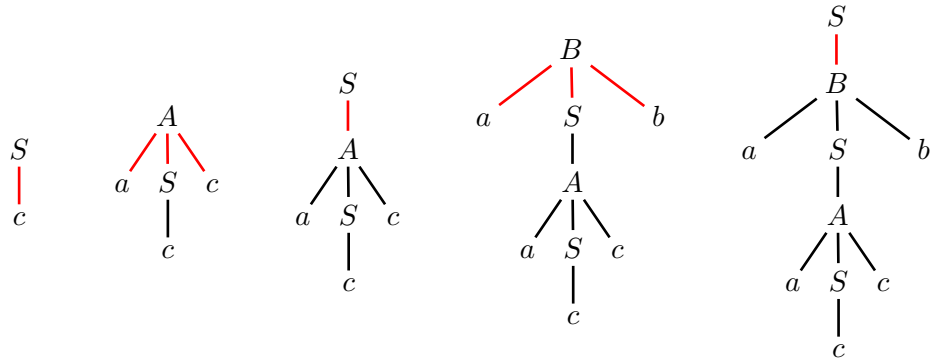
The pilot is clearly free of any conflicts, therefore grammar G is ELR (1).

(c) Simulation of the bottom-up analysis of the (valid) string $a\,a\,c\,c\,b$:

ELR simulation table – string $a\,a\,c\,c\,b$ – number of rows not significant

stack of m-states and grammar symbols	*input left to scan*	*move executed*
I_0	$a\ a\ c\ c\ b\ \dashv$	initial configuration
$I_0\ a\ I_2$	$a\ c\ c\ b\ \dashv$	terminal shift: $I_0 \xrightarrow{a} I_2$
$I_0\ a\ I_2\ a\ I_7$	$c\ c\ b\ \dashv$	terminal shift: $I_2 \xrightarrow{a} I_7$
$I_0\ a\ I_2\ a\ I_7\ c\ I_6$	$c\ b\ \dashv$	terminal shift: $I_7 \xrightarrow{c} I_6$
$I_0\ a\ I_2\ a\ I_7$	$c\ b\ \dashv$	**reduction** 1: $c \rightsquigarrow S$
$I_0\ a\ I_2\ a\ I_7\ S\ I_8$	$c\ b\ \dashv$	nonterminal shift: $I_7 \xrightarrow{S} I_8$
$I_0\ a\ I_2\ a\ I_7\ S\ I_8\ c\ I_9$	$b\ \dashv$	terminal shift: $I_8 \xrightarrow{c} I_9$
$I_0\ a\ I_2$	$b\ \dashv$	**reduction** 2: $a\ S\ c \rightsquigarrow A$
$I_0\ a\ I_2\ A\ I_6$	$b\ \dashv$	nonterminal shift: $I_2 \xrightarrow{A} I_6$
$I_0\ a\ I_2$	$b\ \dashv$	**reduction** 3: $A \rightsquigarrow S$
$I_0\ a\ I_2\ S\ I_3$	$b\ \dashv$	nonterminal shift: $I_2 \xrightarrow{S} I_3$
$I_0\ a\ I_2\ S\ I_3\ b\ I_5$	$\dashv$	terminal shift: $I_3 \xrightarrow{b} I_5$
I_0	$\dashv$	**reduction** 4: $a\ S\ b \rightsquigarrow B$
$I_0\ B\ I_1$	$\dashv$	nonterminal shift: $I_0 \xrightarrow{B} I_1$
I_0	$\dashv$	**reduction** 5: $B \rightsquigarrow S$
initial stack symbol	*empty input*	*reduction to axiom*
acceptance condition verified		

tree building steps (bottom-up)



red: 1 red: 2 red: 3 red: 4 red: 5

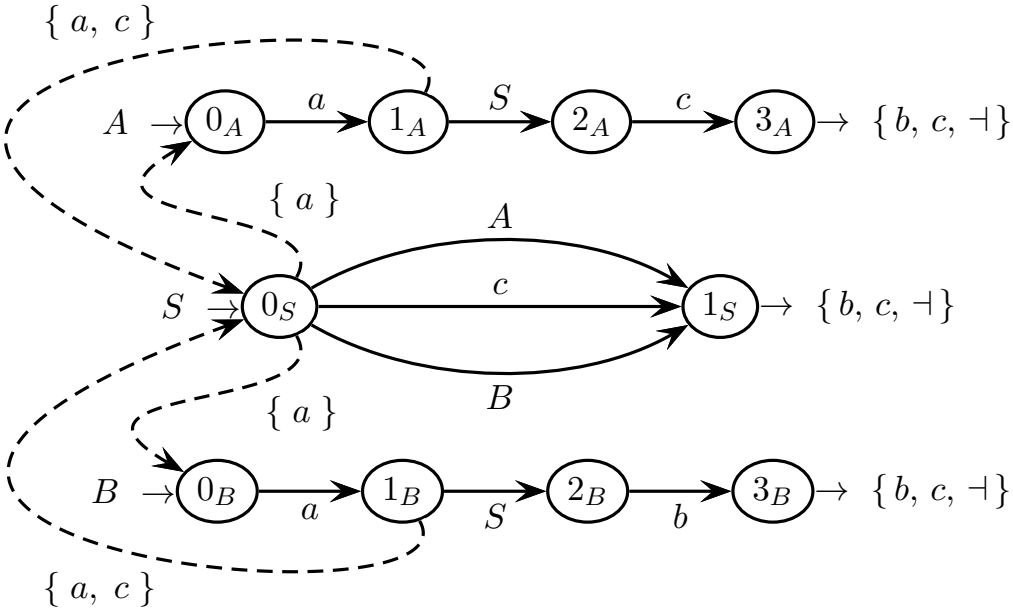
One can compact the sequences of two or more (non)terminal shifts, for instance:

I_0	$a\ a\ c\ c\ b\ \dashv$	initial configuration
$I_0\ a\ I_2\ a\ I_7\ c\ I_6$	$c\ b\ \dashv$	terminal shifts: $I_0 \xrightarrow{a\,a\,c} I_6$

$I_0\ a\ I_2\ a\ I_7$	$c\ b\ \dashv$	immediately after reduction 1
$I_0\ a\ I_2\ a\ I_7\ S\ I_8\ c\ I_9$	$b\ \dashv$	(non)terminal shifts: $I_7 \xrightarrow{S\,c} I_9$

to shorten the list of moves.

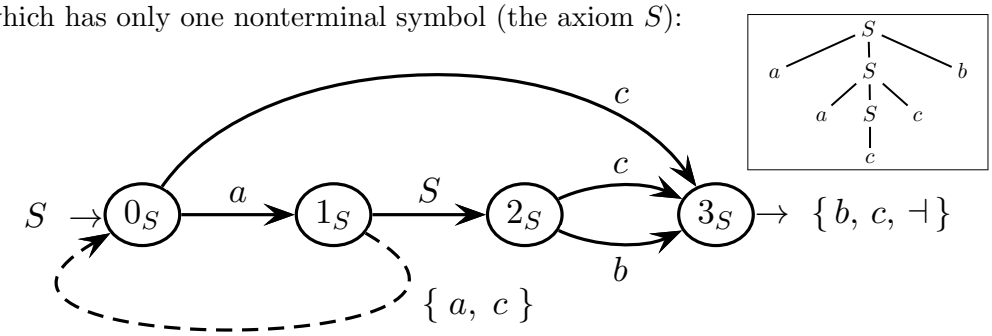
(d) Here are all the guide sets of grammar G (on all the call arcs and exit arrows):



The guide sets on the two call arcs from state 0_S are not disjoint; in fact they coincide. Therefore grammar G is not ELL (1). This is coherent with the observation that the pilot does not have the STP.

(e) Both the machines M_A and M_B of the net of grammar G, after their initial arcs labeled a, i.e., $0_A \xrightarrow{a} 1_A$ and $0_B \xrightarrow{a} 1_B$, have a recursive call to machine M_S. Thus for any look-ahead window of size $k \geq 1$, both guide sets on the call arcs from state 0_S include the string a^k. Therefore such sets are not disjoint, and grammar G is not ELL (k) for any $k \geq 1$.

(f) Here is an equivalent ELL (1) grammar G', represented as one machine M'_S, which has only one nonterminal symbol (the axiom S):



The two machines M_A and M_B separately generate the two very similar nested structures $a^n\, S\, c^n$ and $a^n\, S\, b^n$, respectively (with $n \geq 1$), and play the same role in the original net of grammar G. Here the machines M_A and M_B are unified into one axiomatic machine M'_S, which reads indifferently letter c or letter b.

Alternatively, grammar G' could be represented as one EBNF rule, which structurally corresponds to the machine above:

$$S \to a\ S\ \big(\ c\ \mid\ b\ \big)\ \mid\ c$$

This self-embedding rule generates strings like $a^n\ S\ y^n$, with $n \geq 0$, where each letter y may be c or b, and ends with $a^n\ c\ y^n$ (see the framed sample tree above).

(g) Here is the recursive-descent top-down analyzer of grammar G'. Actually it consists of just one procedure, for nonterminal S (the axiom). The procedure maps the structure of the machine M_S of G' (variable cc is global):

```
char cc                                                  // current character

procedure S
begin
    switch cc do                                           // select guide
        case a do                                        // branch a -- 0_S
            cc := next                                         // go to 1_S
            if cc ∈ { a, c } then                              // state 1_S
                call S                                         // go to 2_S
                if cc ∈ { b, c } then                          // state 2_S
                    cc := next                                 // go to 3_S
                    if cc ∈ { b, c, ⊣ } then return            // state 3_S
                    else error                               // error in 3_S
                else error                                   // error in 2_S
            else error                                       // error in 1_S
        case c do                                        // branch ⊣ -- 0_S
            cc := next                                         // go to 3_S
            if cc ∈ { b, c, ⊣ } then return                    // state 3_S
            else error                                       // error in 3_S
```

As usual, the error cases can be grouped at the end of the procedure (the branches of the switch structure are mutually exclusive):

```
procedure S
begin
    switch cc do                                           // select guide
        case a do                                        // branch a -- 0_S
            cc := next                                         // go to 1_S
            if cc ∈ { a, c } then                              // state 1_S
                call S                                         // go to 2_S
                if cc ∈ { b, c } then                          // state 2_S
                    cc := next                                 // go to 3_S
                    if cc ∈ { b, c, ⊣ } then return            // state 3_S
            break                          // do not fall into the next case
        case c do                                        // branch ⊣ -- 0_S
            cc := next                                         // go to 3_S
            if cc ∈ { b, c, ⊣ } then return                    // state 3_S
    error                                          // any syntactic error
```

In this way however, error detection (if it were required) would be less precise.

4 Language Translation and Semantic Analysis 20%

1. Consider the following BNF source grammar G that represents real numbers, with an integer part and a fractional part (axiom R):

$$G \begin{cases} R \rightarrow S\ I\ .\ F \\ S \rightarrow + \\ S \rightarrow - \\ S \rightarrow \varepsilon \\ I \rightarrow I\ d \mid \varepsilon \\ F \rightarrow d\ F \mid \varepsilon \end{cases}$$

The number can be preceded by an optional sign, and can have zero or more digits in both the integer part and the fractional part (terminal d schematizes any digit).

Answer the following questions:

(a) Design a grammar τ_1 that translates the source number so that the sign before the integer part is always present and the fractional part is not empty. Do not change the source grammar. Example:

$$\tau_1\,(d.) = +\,d.0$$

(b) Design a grammar τ_2 that translates the source number so that the integer part includes at least two digits. This may require to modify the source grammar. Example:

$$\tau_2\,(-\,d.d) = -\,0d.d$$

(c) (optional) Verify translation τ_1 by drawing the source and target trees for the translation of the string "$d.$".

Solution

(a) Here is a grammar G_1 for translation τ_1, with source grammar G unchanged:

$$G_1 \begin{cases} R \;\to\; S\ I\ \dot{\div}\ F \\ S \;\to\; \frac{+}{+} \\ S \;\to\; \frac{-}{-} \\ S \;\to\; \frac{\varepsilon}{+} & \text{// specify the sign + if it is missing} \\ I \;\to\; I\ \frac{d}{d} \;\mid\; \frac{\varepsilon}{\varepsilon} \\ F \;\to\; \frac{d}{d}\ F \;\mid\; \frac{\varepsilon}{0} & \text{// add one 0 if none in the fractional part} \end{cases}$$

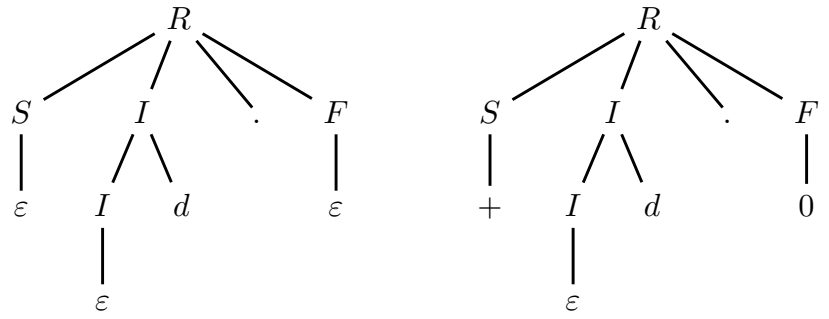
The comments aside highlight where and how the requirements for translation τ_1 are enforced. All the other source terminals are translated without changes.

(b) Here is a grammar G_2 for translation τ_2, by changing the source grammar G:

$$G_2 \begin{cases} R \;\to\; S\ I\ \dot{\div}\ F \\ S \;\to\; \frac{+}{+} \\ S \;\to\; \frac{-}{-} \\ S \;\to\; \frac{\varepsilon}{\varepsilon} \\ \hline I \;\to\; I_1\ \frac{d}{d} \;\mid\; \frac{\varepsilon}{0\,0} & \text{// add two 0's if none in the integer part} \\ I_1 \;\to\; I_2\ \frac{d}{d} \;\mid\; \frac{\varepsilon}{0} & \text{// add one 0 if only one in the integer part} \\ I_2 \;\to\; I_2\ \frac{d}{d} \;\mid\; \frac{\varepsilon}{\varepsilon} \\ \hline F \;\to\; \frac{d}{d}\ F \;\mid\; \frac{\varepsilon}{\varepsilon} \end{cases}$$

As before, the comments aside highlight the crucial points for translation τ_2. Grammar G_2 uses two more nonterminals, called I_1 and I_2, that count the number of digits in the integer part. The two new rules add to the integer part the appropriate number of leading digits 0, to grant a minimal length of two.

(c) Here are the source and destination (or target) syntax trees for the sample translation $\tau_1\,(d.) = +\,d.0$, according to grammar G_1:



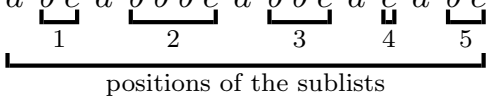
source tree destination (target) tree

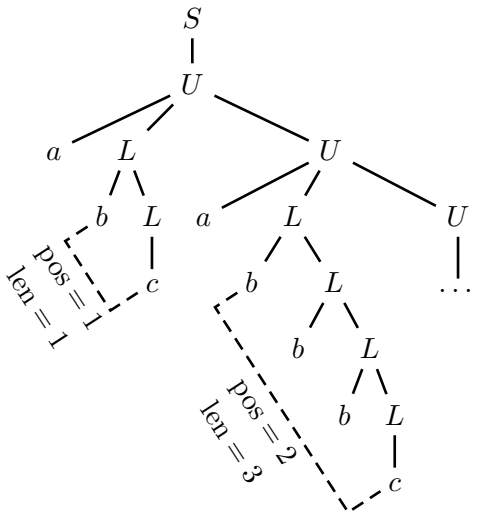
Of course, these two syntax trees can be equivalently unified into one tree.

2. Consider the BNF syntactic support below, which generates a language of two-level lists. The upper-level list (possibly empty) consists of lower-lever sublists, each preceded by a letter a. Each sublist consists of zero or more elements b, and is terminated by a letter c. The terminal and nonterminal alphabets are $\Sigma = \{ a, b, c \}$ and $V = \{ L, U, S \}$ (axiom S):

$$\begin{cases} S \rightarrow U \\ U \rightarrow a\ L\ U & // \text{ upper-level list} \\ U \rightarrow \varepsilon \\ L \rightarrow b\ L & // \text{ lower-level list} \\ L \rightarrow c \end{cases}$$

Example (left: upper-level list of five sublists – right: tree prefix with two sublists):





Answer the following questions (use the tables / trees / spaces on the next pages):

(a) Define an attribute grammar that computes the position and the length, i.e., the number of letters b, of each sublist; see also the example above. For this, use only an integer attribute p (position), right, associated to nonterminal U, and an integer attribute l (length), left, associated to nonterminal L.

(b) By extending the attribute grammar of point (a), define an attribute grammar, of type one-sweep, that checks the property that the length of every sublist is equal to its position. For this, to the attributes p and l already computed, add a boolean attribute e, left, associated to nonterminals U and S, for evaluating the condition. The final decision ($e = true$ means satisfied) must be available in the root node S. Justify that the obtained attribute grammar is one-sweep.

(c) Say and justify if the grammar of point (b) satisfies the L condition.

(d) (optional) For the grammar of point (b), write the semantic procedure of nonterminal U.

table to be filled in for question (a)

#	*syntax*	*semantic functions*
1:	$S_0 \rightarrow U_1$	
2:	$U_0 \rightarrow a\ L_1\ U_2$	
3:	$U_0 \rightarrow \varepsilon$	
4:	$L_0 \rightarrow b\ L_1$	
5:	$L_0 \rightarrow c$	

table to be filled in for question (b)
no need to repeat the functions for attributes p and l

#	*syntax*	*semantic functions*
1:	$S_0 \rightarrow U_1$	
2:	$U_0 \rightarrow a\ L_1\ U_2$	
3:	$U_0 \rightarrow \varepsilon$	
4:	$L_0 \rightarrow b\ L_1$	
5:	$L_0 \rightarrow c$	

Solution

(a) Here are the semantic functions for attributes p (inherited) and l (synthesized):

#	*syntax*	*semantic functions*
1:	$S_0 \rightarrow U_1$	$p_1 := 1$
2:	$U_0 \rightarrow a\ L_1\ U_2$	$p_2 := p_0 + 1$
3:	$U_0 \rightarrow \varepsilon$	
4:	$L_0 \rightarrow b\ L_1$	$l_0 := l_1 + 1$
5:	$L_0 \rightarrow c$	$l_0 := 0$

The inherited attribute p depends only on itself from the parent node, so it is computed strictly top-down. The synthesized attribute l depends only on itself from a child node, so it is computed strictly bottom-up. Therefore this partial grammar is one-sweep.

(b) Here are the semantic functions for attribute e (synthesized). The semantic functions for attributes p and l are already in table (a), and here are unchanged:

#	*syntax*	*semantic functions*
1:	$S_0 \rightarrow U_1$	$e_0 := e_1$

2:	$U_0 \rightarrow a\ L_1\ U_2$	**if** $p_0 = l_1$ **then**
		$e_0 := e_2$
		else
		$e_0 := \mathit{false}$
		endif

3:	$U_0 \rightarrow \varepsilon$	$e_0 := \mathit{true}$
4:	$L_0 \rightarrow b\ L_1$	
5:	$L_0 \rightarrow c$	

One can more compactly write the semantic rule 2 as one assignment $e_0 := e_2 \ \wedge\ (p_0 = l_1)$ or also $e_0 :=\ ?\ p_0 = l_1 : e_2 : \mathit{false}$, in the style of the C language.
Attribute e, synthesized, depends on itself from a child node, on attribute p, inherited, from the same (parent) node, and on attribute l, synthesized, from a child node. Therefore also attribute e satisfies the one-sweep condition, and in conclusion the complete grammar is one-sweep.

(c) For the rule 2 of nonterminal U, the brother graph has two nodes L and U, but no arcs (as there are no dependences between the attributes of the brother nodes L_1 and U_4), therefore it trivially satisfies the L condition. For all the other rules, the brother graph is empty or has only one node, so it also trivially satisfies the L condition. In conclusion, the whole grammar satisfies the L condition.

Furthermore, the syntactic support is clearly LL (1), as the alternative rules 2 and 3 have disjoint guide sets $\{ a \}$ and $\{ \dashv \}$, respectively, and the alternative rules 4 and 5 have disjoint guide sets $\{ b \}$ and $\{ c \}$, respectively. Consequently, for the grammar it would be possible to integrate the syntactic recursive-descent analyzer and the semantic one-sweep analyzer.

(d) Since the attribute grammar of point (b) is one-sweep, it has a one-sweep semantic analyzer. Here are the semantic procedure for nonterminal U and the header of the procedure for nonterminal L, on which procedure U depends. Also the subtree pointer t is passed (the syntax tree must have been entirely pre-built):

```
procedure L (in: t; out: l0)                                  // header only

procedure U (in: t, p0; out: e0)
var l1, p2, e2                  // all local vars of the proper types
begin
    switch node type of U do                                  // select node type
        case U → a L U do                                     // branch U → a L U
            call L (t → L, l1)
            p2 := p0 + 1                                          // update p2
            call U (t → U, p2, e2)
            if p0 = l1 then e0 := e2                              // update e0
            else e0 := false
            return
        case U → ε do                                          // branch U → ε
            e0 := true                                        // initialize e0
            return
```

Here is the same code (only procedure U), by grouping all the return cases at the end (the branches of the switch structure are mutually exclusive):

```
procedure U (in: t, p0; out: e0)
var l1, p2, e2                  // all local vars of the proper types
begin
    switch node type of U do                                  // select node type
        case U → a L U do                                     // branch U → a L U
            call L (t → L, l1)
            p2 := p0 + 1                                          // update p2
            call U (t → U, p2, e2)
            if p0 = l1 then e0 := e2                              // update e0
            else e0 := false
            break                // do not fall into the next branch
        case U → ε do e0 := true          // branch U → ε -- init e0
    return                                                        // any return
```

Similar procedures can be written for nonterminals L and S.

For teaching purposes, in addition to the exercise. Since the attribute grammar of point (b) is one-sweep and satisfies the L condition, see point (c), it has an integrated recursive-descent syntactic-semantic analyzer. Here is the (parameter-less) procedure U of the pure recursive-descent syntactic analyzer. Variable *cc* represents the current character. It is a global variable, as it is used by all the procedures:

```
char cc                                              // current character
procedure U
begin
    switch cc do                                          // select guide
        case a do                                      // branch U → a L U
            cc := next
            if cc ∈ { b, c } then
                call L
                if cc ∈ { a, ⊣ } then
                    call U
                    if cc = ⊣ then return
                    else error                  // incorrect prospect of U
                else error                         // incorrect guide of U
            else error                             // incorrect guide of L
        case ⊣ do                                          // branch U → ε
            if cc = ⊣ then return
            else error                          // incorrect prospect of U
```

More compactly, here is the same code with all the error cases grouped at the end (the branches of the switch structure are mutually exclusive):

```
procedure U
begin
    switch cc do                                          // select guide
        case a do                                      // branch U → a L U
            cc := next
            if cc ∈ { b, c } then
                call L
                if cc ∈ { a, ⊣ } then
                    call U
                    if cc = ⊣ then return
            break                  // do not fall into the next branch
        case ⊣ do return                                   // branch U → ε
    error                                        // any syntactic error
```

Of course, the return cases remain separate.

Here is eventually the integrated procedure for nonterminal U, for simplicity with all the error cases grouped at the end (the branches of the switch structure are mutually exclusive), so that the syntactic and semantic procedures can be just overlapped. The subtree pointer is not passed, as the tree remains implicit:

```
procedure U (in: p0; out: e0)
var l1, p2, e2                          // local vars of the proper types
begin
    switch cc do                                          // select guide
        case a do                                      // branch U → a L U
            cc := next
            if cc ∈ { b, c } then
                call L (l1)
                if cc ∈ { a, ⊣ } then
                    p2 := p0 + 1                                // update p2
                    call U (p2, e2)
                    if cc = ⊣ then
                        if p0 = l1 then e0 := e2                // update e0
                        else e0 := false
                        return
            break                  // do not fall into the next branch
        case ⊣ do                                         // branch U → ε
            e0 := true                                    // initialize e0
            return
    error                                          // any syntactic error
```

The integrated procedure parses the input and altogether computes the attributes. Similar integrated procedures can be given for nonterminals L and S.